

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I M.KASIF JALAL (192424214) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: KASIF

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming

computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Website Clicks, Mobile Interactions and Transactions

1. Data ingestion

- Website clicks, mobile events, and transactions generate billions of records.
- Apache Kafka is likely used for high-speed event ingestion.
- Flume / Logstash may collect application and server logs.

2. Distributed storage

- HDFS or cloud object storage such as Amazon S3/Azure Data Lake stores massive datasets.
- Data is distributed across multiple nodes for scalability and fault tolerance.

3. Data separation

- Structured data: customer IDs, transactions, prices → stored in relational databases/data warehouses.
- Semi-structured data: JSON clickstreams, application logs → stored in HDFS/Data Lake.
- Metadata/catalog systems help identify schemas and data locations.

4. Batch processing

- $\text{Daily sales} = \text{SUM}(\text{transaction_amount})$
- $\text{Average purchase} = \text{Total Sales} / \text{Number of Transactions}$

5. Stream processing

- Kafka Streams, Apache Flink, or Spark Streaming process clicks and transactions in real time.
- Example: detect a sudden increase in product views within seconds.

6. Scalability and partitioning

- Data is partitioned by customer ID, date, region, or transaction ID.
- More nodes can be added horizontally when data volume increases.
- Replication provides fault tolerance.

7. Analytics and dashboards

- Processed data is loaded into a data warehouse or analytical database.
- Power BI, Tableau, or Grafana connects to the analytics layer.

2. E-Commerce Personalization and Recommendation

1. Event streaming

- Product views, cart additions, checkout abandonment, and payments generate events.
- Apache Kafka acts as the distributed event-streaming platform.

2. Customer profiling

- Customer profiles can be stored in MongoDB, Cassandra, DynamoDB, or HBase.

- Profile information includes:

- Customer ID
- Previous purchases
- Browsing history
- Preferences
- Location

3. Session and clickstream synchronization

- Real-time events are sent through Kafka.
- Flink or Spark Streaming joins current sessions with customer profiles.
- Transaction records are synchronized with browsing events using customer/session IDs.

4. Caching

- Redis can store frequently accessed information such as:
 - Product details
 - Customer recommendations
 - Shopping-cart information
- This reduces database response time.

5. Distributed query processing

- Presto/Trino, Spark SQL, or Hive can query large datasets.
- Example:

Conversion Rate = Completed Orders / Total Checkout Sessions $\times$ 100

3. Big Data Healthcare System

1. Data sources

- Electronic health records
- Diagnostic images
- Wearable sensors

2. Hybrid database architecture

- Relational databases: patient IDs, appointments, prescriptions, billing.
- NoSQL databases: wearable and semi-structured patient data.
- Object storage/Data Lake: X-rays, CT scans, MRI images and documents.
- This combination supports both structured and unstructured data.

3. Data integration

- Apache NiFi, Kafka, ETL tools, and healthcare interoperability standards can connect hospitals and laboratories.
- HL7/FHIR can be used for exchanging healthcare information.

4. Streaming

- Wearable devices continuously generate heart rate, temperature, oxygen level, etc.
- Kafka + Spark Streaming/Flink can process these events in real time.
- Abnormal readings can trigger alerts.

5. Applications

- Clinical decision support
- Disease prediction
- Patient monitoring
- Population health analysis
- Hospital resource planning

4. Smart City Big Data Platform

Probable Architecture

1. Data sources

- Traffic sensors
- CCTV metadata
- Weather stations
- GPS
- Public transport systems

2. Edge computing

- Sensors and edge devices perform initial processing close to the data source.
- For example, CCTV cameras can generate vehicle-count metadata instead of continuously sending raw video.

3. Messaging system

- Apache Kafka or MQTT can transport sensor events.
- Kafka partitions traffic events by location/road/region.

4. Centralized storage

- HDFS or cloud Data Lake stores historical data.
- Time-series databases such as InfluxDB can store sensor measurements.
- PostGIS or another spatial database can support location-based queries.

5. Security

- Encryption protects data.
- Authentication and RBAC control access.
- Network segmentation protects critical infrastructure.
- Audit logs track access and system activities.

6. Visualization

- Grafana, Power BI, Tableau, or GIS dashboards can display:
 1. Live traffic maps
 2. Congestion levels
 3. Bus locations
 4. Weather conditions
 5. Traffic predictions

5. Healthcare Analytics: Records + Wearables + Labs

1. Data sources

- Patient records
- Wearable devices
- Laboratory reports
- Appointment systems
- Hospital/clinic databases

2. Hybrid storage

- SQL databases: structured patient, appointment, and billing information.
- NoSQL: wearable and semi-structured data.
- Data Lake/Object storage: medical reports and documents.
- This allows different data types to coexist.

3. ETL and interoperability

- Apache NiFi, Talend, Kafka, or cloud ETL tools can collect and transform data.
- HL7/FHIR can support interoperability between healthcare systems.
- ETL stages:
 1. Extract
 2. Clean
 3. Transform
 4. Load

4. Distributed processing

- Apache Spark/Hadoop processes millions of patient records.
- Example:

Average Risk Score = Sum of Patient Risk Scores / Number of Patients

5. Data governance

- Encryption protects sensitive information.
- RBAC limits access.
- Authentication verifies users.
- Audit logs record data access.